# CS 240 - Lab 4

TAs: Sujan & Aditya

Spring 2025

Welcome to the fourth graded lab of this course!!

## Instructions:

- This lab has 4 questions. The first Question is the graded inlab. While the fourth question will be part of the graded outlab.
- Create a folder on your Desktop with the following directory structure. The folder must contain only the two specified files:

```
submission_<roll_number>/
|-- q1.py
```

- Once you have completed the work, open the terminal, type the following command, and then call a TA: `check-cs240`
- The TA will run `submit-cs240` to submit your work to the server. **It is your responsibility to ensure your work is submitted.** If you fail to inform us to submit your work, you will not be graded.

# Q1. Multi-Class Logistic Regression for Imbalanced Classes

In this section, you will develop a multi-class classification system to handle a dataset with imbalanced classes, specifically *Prior Probability Shift*. Prior probability shift occurs when the class prior distributions differ between training and test data, while the class-conditional distributions remain unchanged:

$$P_{\text{train}}(y) \neq P_{\text{test}}(y), \qquad P_{\text{train}}(x \mid y) = P_{\text{test}}(x \mid y).$$

In this setting, the appearance or feature distribution of each class remains the same, but the relative frequency of classes changes. As a result, a classifier trained under the training distribution becomes biased toward the training class proportions.
To correct for this mismatch, importance weighting can be applied. Each training example with label $y$ is assigned a weight

$$w(y) = \frac{P_{\text{test}}(y)}{P_{\text{train}}(y)},$$

so that the effective training distribution matches the test-time class prior.

In this inlab, you will implement Importance Weighting within a One-Vs-Rest (OvR) framework to deal with Prior Probability Shift. You will implement this in three distinct stages.

## Task 1 Evaluation Metrics

Before building the classifier, you must implement the evaluation metrics to assess performance. For multi-class classification, these metrics are often calculated for each class individually (one-vs-rest).

- **Precision:** The fraction of relevant instances among the retrieved instances.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** The fraction of relevant instances that were retrieved.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** The harmonic mean of Precision and Recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

In case you forgot, here are the relevant definitions (Note that these are for a One vs Rest classifier trying to identify instances of a particular class).

- **TP**: We predict that it belongs to the class and it actually belongs to the class
- **FP**: We predict that it belongs to the class and it actually does not belong to the class
- **FN**: We predict that it does not belong to the class while it actually does belong to the class.

## Task 2 The Class-Weighted Binary Classifier

The core model is a Binary Logistic Regression model. Standard logistic regression assumes all samples are equally important. To handle prior probability shift, you must modify the objective function to accept **Importance Weights**.

**Weighted Binary Cross-Entropy Loss** For a set of $M$ samples, where each sample $i$ has an associated importance weight $w^{(i)}$, the cost function $J(\theta)$ is the weighted average of the individual losses:

$$J(\theta) = \frac{-1}{\sum_{i=1}^{M} w^{(i)}} \sum_{i=1}^{M} w^{(i)} \left[ y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right]$$

where $h_\theta(x) = \sigma(\theta^T x)$ is the sigmoid activation function.

Implement the functions in the `BaseLogisticClassifier` class to support these weighted operations.

## Task 3 Multi-Class Wrapper (One-Vs-Rest)

The final task is to implement the `OneVsRestClassifier`, which manages $K$ binary classifiers. This is where the core **Prior Probability Shift** logic is applied. You must calculate the correct importance weights ($w$) for each binary sub-problem before passing them to the base classifier.

**Importance Sampling Logic** For a specific binary classifier distinguishing Class $k$ (Positive) from the Rest (Negative), the weight $w$ for a sample depends on the ratio of its target probability to its source probability:

$$w = \frac{P_{test}}{P_{train}}$$

Your `_get_binary_weights` function should calculate weights as follows:

1. **Calculate Training Priors ($P_{train}$):** Compute the empirical fraction of positive (1) and negative (0) samples in the current training batch.

$$P_{train}(1) = \frac{\text{Count}(y = k)}{N}, \quad P_{train}(0) = 1 - P_{train}(1)$$

2. **Retrieve Test Priors ($P_{test}$):** Use the provided `test_ratios` dictionary to find the expected probability of Class $k$.

$$P_{test}(1) = \text{test_ratios}[k], \quad P_{test}(0) = 1 - P_{test}(1)$$

3. **Compute Binary Weights:**

$$w_{pos} = \frac{P_{test}(1)}{P_{train}(1)}, \quad w_{neg} = \frac{P_{test}(0)}{P_{train}(0)}$$

In the `fit` method of `OneVsRestClassifier`, loop through all classes. For each class, generate the binary targets, calculate the corresponding $w_{pos}$ and $w_{neg}$ (if `mode='weighted'`), and train a `BaseLogisticClassifier` using those weights.

**Important Instructions**

1. Please strictly follow the provided directory structure. Any deviation will result in a grade of 0.
2. All functions in q1.py are autograded. Ensure your code is free of syntax errors and runtime exceptions. A single error in a function may cause the autograder to fail for that specific task.
3. You can use debug.py for testing and debugging your code. You can print intermediate values or comment out sections to isolate issues.
4. You are only allowed to use NumPy. The import of any other library is not allowed.
5. Strictly adhere to the argument types, return types, and shapes specified in the docstrings. The autograder relies on exact matches for matrix dimensions (e.g., ensuring a return shape of (N,) vs (N, 1)).

## Q2. Bayesian Spam Filtering

In this problem, you will build a Naive Bayes classifier to detect spam SMS messages. You will utilize the **SMS Spam Collection** dataset, which contains 5,574 messages tagged as `ham` (legitimate) or `spam`. You must implement the probabilistic framework described in the reference material without relying on external classification libraries like `sklearn`.

### Part 1: The Preprocesscessing Pipeline

Raw text data is noisy and high-dimensional. Before applying probabilistic models, you must clean the dataset to improve signal-to-noise ratio.

1. Preprocess the data to make the data simple for further modelling. Here are some techniques
   - Convert all characters to lowercase.
   - Stop Word Removal: Remove common English words(e.g., "the", "is", "at").

   It is advised to have a look at other preprocessing techniques for the data.
2. Visualise the most frequent words for both the `spam` and `ham` classes after preprocessing.

### Part 2: The Bayesian Framework

The core of the filter relies on Bayes' Theorem. We aim to calculate $P(S|W)$, the probability that a message is spam ($S$) given a specific word ($W$) appears in it.

1. **The Unbiased Assumption:** Derive the simplified formula for $P(S|W)$ assuming that the prior probabilities of receiving spam and ham are equal ($P(S) = P(H) = 0.5$).

$$P(S|W) = \frac{P(W|S)}{P(W|S) + P(W|H)} \tag{1}$$

   where $P(W|S)$ is the frequency of the word in spam messages relative to all messages containing that word.
2. **Feature Probabilities:** Using your training set (70% of the data), calculate the individual "spaminess" probabilities for every unique word in your vocabulary. Store these in a lookup table or dictionary.

### Part 3: Handling Rare Words and Corrections

A major issue in Naive Bayes is handling words with low frequencies or words never seen during training. If a word has $P(W|S) = 0$ or $P(W|H) = 0$, it can destabilize the classifier.

1. **The Zero Frequency Problem:** A major challenge in Naive Bayes is handling words that appear in the test set but were never (or rarely) seen during training. In standard probability estimation, these result in a probability of 0, which can cause the entire classifier to fail (floating point underflow or zeroing out the entire product).
2. **Smoothing Techniques:** To solve this, you must "smooth" the probability estimates to ensure no probability is ever exactly 0 or 1. Read up on the following techniques:
   - **Laplace Smoothing** (also known as Add-One Smoothing).
   - **The m-estimate** (referred to as "Corrected Probability" in some literature).

### Part 4: Combining Probabilities (The "Naive" Step)

Naive Bayes assumes word independence. To classify a message containing words $w_1, w_2, \ldots, w_N$, we combine individual word probabilities.

1. **Floating Point Underflow:** Multiplying many small probabilities can result in floating-point underflow . How can this be avoided?
2. Using the above quantity, make the final classification whether the message is spam or ham ?

## Part 5: Metrics

Test your filter on the remaining 30% of the dataset.

1. Calculate the following metrics:
   - **Spam Precision:** $\frac{TP}{TP+FP}$
   - **Spam Recall:** $\frac{TP}{TP+FN}$
   - **F-Score:** The harmonic mean of precision and recall.
2. Analysis: Your model likely identifies `ham` more accurately than `spam` (higher recall for ham). Explain why this discrepancy exists based on the class imbalance in the dataset. How would you handle this?

# Q3. Problem 3: Gaussian Discriminant Analysis (GDA)

In this problem, you will apply Generative Classifiers to a real-world Remote Sensing task: classifying land cover types from satellite imagery. You will perform a forensic analysis of the model's performance, exploring feature selection, distance metrics, and the Bias-Variance tradeoff.

### Dataset

The `Statlog (Landsat Satellite)` dataset.

- **Classes ($K = 6$):** Red Soil (1), Cotton Crop (2), Grey Soil (3), Damp Grey Soil (4), Soil with Veg. Stubble (5), Mixture (7).
- **Features ($D = 36$):** Multi-spectral values (4 bands) for a $3 \times 3$ pixel neighborhood. Features are flattened: Indices $0 - 3$ are Top-Left, $16 - 19$ are Center, etc.

### Preliminaries: Metrics & Definitions

Before starting, ensure you implement or utilize the following metrics for your analysis:

- **Precision ($P_k$):** $\frac{TP_k}{TP_k+FP_k}$ (Accuracy of positive predictions for class $k$).
- **Recall ($R_k$):** $\frac{TP_k}{TP_k+FN_k}$ (Coverage of actual positives for class $k$).
- **F1-Score:** $2 \cdot \frac{P_k \cdot R_k}{P_k+R_k}$ (Harmonic mean, useful for imbalanced classes).
- **Mahalanobis Distance:** A measure of distance between a point $x$ and a distribution (or mean $\mu$) that accounts for the covariance structure $\Sigma$:

$$D_M(x, \mu) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)} \tag{2}$$

### Task 1 The Geometry of Land Cover

We begin by inspecting the learned parameters to verify physical intuition. Train an LDA and QDA model on the full dataset.

1. **Visualizing Means:** visualise the learned Mean Vectors and covariance matrices for each class
2. **Distance Analysis:** Calculate the distance between the mean vectors to get a prior belief regarding which classes would be hardest to classify and which classes would be toughest to distinguish each other.
   - Compute the **Euclidean Distance**: $||\mu_1 - \mu_3||_2$.
   - Compute the **Mahalanobis Distance** using the LDA pooled covariance: $\sqrt{(\mu_1 - \mu_3)^T \Sigma_{pooled}^{-1} (\mu_1 - \mu_3)}$.
3. **Observation:**
   - Which of the above distance metric would you consider a better metric?
   - Try stretching a feature, i.e multiplying it by a factor say 1000 to see the classic distance metric breakdown

### Task 2 Feature Selection

The dataset uses a $3 \times 3$ pixel grid (spatial context). Let's determine if this added complexity helps or hurts the model.

1. **Experiment:** Create 4 feature subsets and train both LDA and QDA on each:

- **1-Pixel (Center):** 4 features (Indices 16-19).
- **4-Pixels (Corners):** 16 features.
- **5-Pixels (Cross):** 20 features.
- **6-Pixels (Any 2*3 or 3*2 contiguous pixels)** 24 features
- **9-Pixels (Full Grid):** 36 features.

2. **Plot:** Generate a line graph of **Test Accuracy vs. Number of Pixels** for both models.
3. **Observation:**
   - Does QDA performance improve monotonicially as you add pixels, or does it degrade at 9-Pixels?
   - Relate this to the **Curse of Dimensionality**: QDA estimates $K \times D \times D$ parameters. How does increasing $D$ affect the variance of the estimate?

### Task 3 Forensic Performance Analysis

1. **Full Report:** Using the "Full Grid" QDA model, report the Precision, Recall, and F1-Score for each of the 6 classes.
2. **Confusion Matrix:** Plot the Confusion Matrix as a heatmap.
3. **Sample Size Simulation:** Run a simulation varying training size $N \in \{100, 200, \ldots, 4000\}$. Plot Accuracy vs $N$ to find the "Crossover Point".
4. **Observation:**
   - **Hardest Class:** Based on the F1-Scores, which class is the hardest to identify? Verify it with your priors from task 1
   - **Confusion Pair:** Look at the Confusion Matrix. Which pair of classes is most frequently confused ?
   - **Bias-Variance:** At $N = 100$, LDA likely beats QDA. Why does the "simpler" model win when data is scarce?

## Q4. Large Scale Generative Classifiers

Consider a multi-class classification problem with input space $\mathbb{R}^d$ and label space $\{1, \ldots, K\}$, which maybe possibly be very large. We make use of generative classifiers to try and tackle this problem. Along the way, we observe certain issues with our models and try to think of ways to solve them.

**Note:** Task 1 will be evaluated on data akin to train1.csv, and Tasks 2,3 will be evaluated on data similar to train2.csv.

### Class-Conditional Generative Model

Assume that the joint distribution over inputs and labels is given by

$$p(x, y = k) = p(x \mid y = k)\, \pi_k,$$

where the class priors satisfy $\sum_{k=1}^{K} \pi_k = 1$.
We model the class-conditional densities as Gaussians with shared covariance:

$$p(x \mid y = k) = \mathcal{N}(x \mid \mu_k, \Sigma), \qquad \Sigma \in \mathbb{R}^{d \times d},\ \Sigma \succ 0.$$

### Posterior Inference and Softmax Representation

By Bayes' rule, the posterior distribution over labels is

$$p(y = k \mid x) = \frac{p(x \mid y = k)\pi_k}{\sum_{j=1}^{K} p(x \mid y = j)\pi_j}.$$

Expanding the Gaussian log-density,

$$\log p(x \mid y = k) = -\frac{1}{2}(x - \mu_k)^\top \Sigma^{-1}(x - \mu_k) - \frac{1}{2}\log|\Sigma| + C,$$

and collecting terms that depend on the class index $k$, we obtain

$$\log p(y = k \mid x) = (\Sigma^{-1}\mu_k)^\top x - \frac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k + \log \pi_k - \log Z(x),$$

where

$$Z(x) = \sum_{j=1}^{K} \exp(w_j^\top x + b_j).$$

Thus the posterior can be written in the softmax form

$$p(y = k \mid x) = \frac{\exp(w_k^\top x + b_k)}{\sum_{j=1}^{K} \exp(w_j^\top x + b_j)},$$

with parameters

$$w_k = \Sigma^{-1}\mu_k, \qquad b_k = -\frac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k + \log \pi_k.$$

### Task 1 Parameter Estimation

Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ drawn i.i.d. from the joint distribution $p(x, y)$, consider the conditional log-likelihood

$$\mathcal{L}(W, b) = \sum_{i=1}^{N} \log p(y_i \mid x_i).$$

This objective is maximized with respect to $\{w_k, b_k\}_{k=1}^{K}$. We will be training this objective using mini-batch gradient descent. At each iteration, a mini-batch of size $m$ is drawn from the dataset,

$$\{(x_i, y_i)\}_{i=1}^{m},$$

and gradients are computed using only this subset.
Suppose the learned parameters $\{w_k, b_k\}$ satisfy

$$w_k = \Sigma^{-1}\mu_k \quad \text{for all } k.$$

As a side task, recover the $\pi_k$, the prior probabilities and the covariance matrix $\Sigma$, compare them against the empirical probabilities of each class in the dataset. In all of the equations, use the sample class mean as an estimate of the actual Gaussian mean. Notice, how in this form of estimation the prior knowledge is built into the model, and we do not face issues such as class imbalances.

### Why Approximate Normalization Is Required

Exact posterior inference under the class-conditional generative model requires evaluating the normalization term

$$Z(x) = \sum_{j=1}^{K} p(x \mid y = j)\pi_j.$$

When the number of classes $K$ is large, or when evaluating $p(x \mid y)$ is itself expensive, this computation becomes the dominant cost in both training and inference. The difficulty arises solely from the normalization term; the unnormalized scores $p(x \mid y)\pi_y$ are typically easy to compute. This motivates methods that either approximate the normalization term or avoid computing it altogether.

### Task 2 Importance Sampling for Posterior Approximation

Since computing the normalization is not tractable over a large number of classes, we try to find an unbiased estimate of it. This is known as importance sampling (not to be confused with what was done in the inlab). Let $q(y)$ be a proposal distribution over class labels such that $q(y) > 0$ whenever $\pi_y > 0$. The normalization constant can be expressed as an expectation:

$$Z(x) = \sum_{j=1}^{K} \frac{p(x \mid y = j)\pi_j}{q(j)} q(j) = \mathbb{E}_{y \sim q}\left[\frac{p(x \mid y)\pi_y}{q(y)}\right].$$

Given $M$ i.i.d. samples $\{y_m\}_{m=1}^{M} \sim q(y)$, define the Monte Carlo estimator

$$\hat{Z}(x) = \frac{1}{M} \sum_{m=1}^{M} \frac{p(x \mid y_m)\pi_{y_m}}{q(y_m)}.$$

The posterior distribution is then approximated by

$$p(y \mid x) \approx \frac{p(x \mid y)\pi_y}{\hat{Z}(x)}.$$

**Importance Sampling Algorithm** For each input $x$:

1. Sample class indices $y_1, \ldots, y_M \sim q(y)$.
2. Compute unnormalized weights

$$w_m = \frac{p(x \mid y_m)\pi_{y_m}}{q(y_m)}.$$

3. Estimate the normalization constant

$$\hat{Z}(x) = \frac{1}{M} \sum_{m=1}^{M} w_m.$$

4. Approximate the posterior using

$$p(y \mid x) \approx \frac{p(x \mid y)\pi_y}{\hat{Z}(x)}.$$

The estimator

$$\hat{Z}(x) = \frac{1}{M} \sum_{j=1}^{M} \frac{p(x \mid y_j)\pi_{y_j}}{q(y_j)}, \quad y_j \sim q(y),$$

is recomputed on the fly during training.
At each mini-batch update:

- a fresh set of samples $\{y_j\}_{j=1}^{M}$ is drawn from the proposal distribution $q(y)$,
- the approximate normalization $\hat{Z}(x_i)$ is evaluated for each $(x_i, y_i)$ in the mini-batch,
- gradients are taken with respect to the resulting stochastic objective.

We will be working with the uniform distribution on the $K$ classes, you can change this and try to play around with the distribution which gives the best results.

## Task 3 Noise Contrastive Estimation

Importance sampling approximates the normalization constant explicitly. Noise Contrastive Estimation avoids computing it by reformulating density estimation as a binary classification problem. Essentially, a good model should be able to distinguish true in distribution samples from those generated from a noisy distribution.
Let $p_m(x, y)$ denote the true (unknown) data-generating distribution over input–label pairs. Consider an unnormalized model

$$p_\theta(x, y) = \exp(f_\theta(x, y) - c),$$

where $f_\theta$ is a parametric scoring function and $c \in \mathbb{R}$ is a scalar parameter representing $\log Z_\theta$.
Let $q(x, y)$ be a known noise distribution. Define a binary random variable

$$D = \begin{cases} 1 & (x, y) \sim p_m(x, y), \\ 0 & (x, y) \sim q(x, y). \end{cases}$$

Assume that for each data sample, $k$ noise samples are drawn. By Bayes' rule,

$$P_\theta(D = 1 \mid x, y) = \frac{p_\theta(x, y)}{p_\theta(x, y) + k\, q(x, y)} = \frac{\exp(f_\theta(x, y) - c)}{\exp(f_\theta(x, y) - c) + k\, q(x, y)}.$$

Equivalently,

$$P_\theta(D = 1 \mid x, y) = \sigma(f_\theta(x, y) - c - \log kq(x, y)), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

**Noise Contrastive Estimation Algorithm**

1. Draw data samples $\{(x_i, y_i)\}_{i=1}^{N} \sim p_m(x, y)$.
2. The noise distribution is chosen as a factorized distribution of the form

   $$q(x, y) = q(x)\, q(y),$$

   where $q(y)$ is a fixed categorical distribution over class labels and $q(x)$ is a Gaussian distribution in $\mathbb{R}^d$.

   Specifically, $q(x)$ is taken to be a multivariate Gaussian with mean and covariance estimated from the training data,

   $$q(x) = \mathcal{N}(x \mid \mu_q, \Sigma_q),$$

   and $q(y)$ is chosen to be uniform over $\{1, \ldots, K\}$ unless stated otherwise.

3. For each data sample, draw $k$ noise samples

$$(\tilde{x}_{i,1}, \tilde{y}_{i,1}), \ldots, (\tilde{x}_{i,k}, \tilde{y}_{i,k}) \sim q(x, y).$$

4. Minimize the objective

$$\mathcal{L}_{\text{NCE}}(\theta, c) = -\sum_{i=1}^{N} \log \sigma(f_\theta(x_i, y_i) - c - \log kq(x_i, y_i)) - \sum_{i=1}^{N} \sum_{j=1}^{k} \log \sigma(-f_\theta(\tilde{x}_{i,j}, \tilde{y}_{i,j}) + c + \log kq(\tilde{x}_{i,j}, \tilde{y}_{i,j}))\,.$$

At each iteration:

- a mini-batch $\{(x_i, y_i)\}_{i=1}^{m}$ is drawn from the dataset,
- for each $(x_i, y_i)$, $k$ noise samples $(\tilde{x}_{i,j}, \tilde{y}_{i,j}) \sim q(x, y)$ are drawn on the fly,
- gradients are computed with respect to both $\theta$ and the normalization parameter $c$ using the resulting mini-batch objective.
- In all cases, parameters are updated according to

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_B(\theta),$$

  where the new objective is now the negative log likelihood

## Task 4 Final Evaluation: Importance Sampling vs Noise Contrastive Estimation

In this final task, you will evaluate and compare the two approximate training methods introduced in this problem: importance sampling and Noise Contrastive Estimation.

**Evaluation Protocol** Let $\{(x_i^{\text{test}}, y_i^{\text{test}})\}_{i=1}^{N_{\text{test}}}$ denote the held-out test dataset. For each trained model, compute predictions

$$\hat{y}_i = \arg\max_k p(y = k \mid x_i^{\text{test}}),$$

and evaluate performance using the following metrics:

- Overall classification accuracy
- Per-class precision, recall, and F1-score
- There will be a small percentage of marks allocated to test accuracy which will be evaluated on a curve.
- You are free to implement your own sampling classes or batch size as well to try and increase accuracy.

Overall accuracy is defined as

$$\text{Accuracy} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbb{I}\big[\hat{y}_i = y_i^{\text{test}}\big]\,.$$